

## Agent

- Agent = 사실상 LLM 하나
- GPT 하나, Claude 하나 같은 모델을 특정 역할로 사용한 것
- 예:
  - FAQ 담당 Agent
  - 인증 담당 Agent
  - 고객정보 조회 Agent

## Workflow

- 여러 Agent를 연결해서 작업 수행
- 하나의 GPT에 모든 걸 시키는 대신 업무를 분업

예시

고객 질문

↓

분류 Agent

↓

FAQ Agent / 인증 Agent / DB 조회 Agent

↓

응답 생성 Agent

↓

고객 응답

- LLM은 생각보다 실수가 많음
- 업무를 잘게 쪼개면 정확도가 올라감
- Hallucination 감소

workflow by FDE

keyword : FDE

FDE(Forward Deployed Engineer)는 고객사 현장에 직접 들어가서 업무를 분석하고 AI 시스템을 설계하는 역할

LLM → LLM(auth), LLM(leg), LLM(fag)

Embedding

iaaS, saaS

## Agentic AI

기존 Workflow와 차이:

Workflow

- 사람이 설계

Agentic AI

- AI가 계획(Planning) 생성
- 실행
- 평가(Evaluation)
- 실패 시 재시도

구조

입력

↓

Planner

↓

Executor

↓

Evaluator

↓

재실행(Loop)

핵심 특징

- 스스로 계획 수립
- 스스로 평가
- 반복 수행(Loop)

## RAG (Retrieval Augmented Generation)

LLM에 회사 문서 1만 페이지를 전부 넣을 수 없음

- Context Window 제한

### 1.문서를 잘게 쪼갬 - Chunk

문서

↓

1KB

↓

1KB

### 2.각 Chunk를 Embedding

텍스트

↓

Embedding Model

↓

Vector

### 3. 질문도 Vector로 변환

질문

↓

Embedding

↓

Vector

### 4. 유사도 계산

주로 Cosine Similarity 사용

가장 비슷한 문서 3~5개 선택 -> 질문 + 관련 문서를 GPT에 같이 넣음

### RAG 단점

- Vector 변환 과정에서 정보 손실
- 완벽하지 않음
- 보통 90점 이상 만들기 어려움

## Embedding

텍스트를 숫자 벡터로 변환하는 것

## Context Window

LLM이 한 번에 읽을 수 있는 최대 입력 크기

예)

- GPT 입력 한도
- Claude 입력 한도

문서가 너무 크면 한 번에 못 넣음

그래서 Chunking , RAG 사용

## LLM 학습 과정

### (1) Pretraining

인터넷의 엄청난 텍스트 사용

### (2) Fine-tuning

QA 형태 학습

Q:대한민국 수도는?

A:서울

이런 데이터로 학습

### (3) RLHF

(Reinforcement Learning from Human Feedback)

사람이 평가

학습

목적

- 욕설 방지
- 위험 답변 방지
- 품질 향상

# Token

LLM이 처리하는 최소 단위

단어와 다름

# Hallucination

AI가 그럴듯한 거짓말을 하는 현상

원인

- 지식 부족
- 추론 실패
- 검색 실패

해결

- RAG
- Workflow
- Agent 분업
- Guardrail

# Fine-Tuning

이미 학습된 모델을 특정 업무용으로 재학습

- 회사 상담 챗봇
- 의료 챗봇
- 법률 챗봇

# API

다른 프로그램의 기능을 가져다 쓰는 인터페이스

GPT API

## 클라이언트 / 서버

클라이언트 - 사용자 측

- 브라우저
- 앱
- 스마트폰

서버 - 서비스 제공 측

- 네이버 서버
- GPT 서버

## Cloud Computing

직접 서버를 두는 대신 외부 데이터센터 사용

대표

- [AWS](#)
- [Microsoft Azure](#)
- 서버 구매 불필요
- 확장 쉬움
- 유지보수 쉬움

## On-Premise

회사 내부 서버에 직접 설치

- 금융

- 공공기관
- 인터넷 없이 동작 가능
- 보안 강함

반대 개념 - Cloud

## Frontend / Backend

**Frontend** 사용자가 보는 화면

- HTML
- CSS
- JavaScript

**Backend** 서버 로직

- Python
- Java
- Node.js

**Full Stack**

Frontend + Backend 모두 가능

## JSON

표준 데이터 교환 형식

# Agile

빠르게 만들고 빠르게 수정하는 개발 방식

예전

설계 → 개발 → 출시

지금

만들기 → 테스트 → 수정 반복

## MVP

(Minimum Viable Product) 최소 기능 제품

- 시장 반응 확인
- 실패 비용 감소

배달앱 전체 개발 x / 주문 기능만 먼저 개발

## B2B / B2C

**B2B** 기업 ↔ 기업

- 보험사 AI 구축
- 기업용 솔루션

**B2C** 기업 ↔ 소비자

- ChatGPT
- 네이버

- 배달앱

## PM(Project Manager)

최근 중요해짐

- 일정 관리
  - 요구사항 정리
  - 고객사 소통
  - 개발팀 소통
- 
- 커뮤니케이션
  - 기술 이해

## Guardrail

AI 안전장치

- 욕설 금지
- 프롬프트 공개 금지
- 개인정보 노출 금지